Predicting Wine Quality

Using Machine Learning

Dataset: UCI Wine Quality Dataset | Source: Cortez et al., 2009

1. Project Overview

Wine quality assessment has traditionally relied on human experts performing sensory evaluation, a process that is inherently subjective, time-intensive, and expensive. This project explores whether machine learning can help support or complement human judgment using measurable physicochemical properties of wine, and machine learning to make sense of them.

The main question we aim to answer is: “Given the physicochemical properties of a wine sample, can we predict whether it will be rated as good quality?” This turns the problem into a binary classification task, one that machine learning is well-suited for.

1.1 The Dataset

We worked with the UCI Wine Quality Dataset, which contains samples from the Vinho Verde region of Portugal. The dataset includes two separate files:

- Red wine: 1,599 samples
- White wine: 4,898 samples
- Combined total: 6,497 wine samples, each described by 11 chemical features

We merged the two (red and white wines) datasets to increase the overall sample size, allowing our models to learn from a richer and more diverse set of observations. This approach helps improve the robustness and generalizability of our results. However, this also introduces a limitation, as the two wine types have inherently different characteristics. To address this, we retained wine type (red vs white) as a feature in the model, ensuring that the model can still differentiate between them when generating final recommendations.

1.2 The Machine Learning Task

The target variable (dependent) is **quality, an expert** score from 0 to 10. We categorized this into two classes:

- Good wine: quality score ≥ 7 (label = 1)
- Not Good wine: quality score < 7 (label = 0)

1.3 Why This Matters

Automating wine quality prediction can be very useful in real life. Wineries could reduce their reliance on expensive expert tasting panels, catch quality issues earlier in production, and gain scientific insight into exactly which chemical factors drive perceived quality. The project also presents an interesting methodological challenge: significant class imbalance, since only about 19.7% of wines in the combined dataset are “Good.”

Fun Fact! Our group likes wine a lot. So this project was especially interesting for us because it gave us a chance to learn more about what contributes to classifying a wine as “good”.

1.4 Notebook Structure

Step	Section Title	Purpose
1	Importing Libraries	Set up all required Python packages
2	Loading the Dataset	Read and inspect both wine CSV files
3	Exploratory Data Analysis (EDA)	Understand distributions, correlations, and class imbalance

4	Data Preparation & Feature Engineering	Merge datasets, engineer new features, prepare data for modeling
5	Model Training	Train and tune four ML classifiers using cross-validation
6	Model Evaluation & Comparison	Evaluate and compare all models on a held-out test set
7	Conclusion	Interpret results, note limitations, and suggest future work

2. Technical Setup & Libraries

Below are all the libraries used and the reason each one was chosen.

Library	Used For
pandas	Data loading, manipulation, and tabular operations
numpy	Numerical operations and array handling
matplotlib	Base plotting library for charts and graphs
seaborn	Statistical visualizations, especially the correlation heatmap
sklearn (scikit-learn)	Model training, cross-validation, preprocessing, and metrics
xgboost	XGBoost classifier one of our four models
imblearn (imbalanced-learn)	SMOTE oversampling to handle class imbalance

A key reproducibility decision: every random process in the notebook uses random_state=42 to ensure that model initializations produce identical results every time the notebook is run.

3. Loading the Dataset

We loaded the two CSV files with semicolons as the delimiter ";", as opposed to the standard comma. After loading, a "type" column was added to distinguish red wine (1) from white wine (0), and then the two datasets were combined into a single dataframe.

```
red_wine = pd.read_csv('winequality-red.csv', sep=';')
white_wine = pd.read_csv('winequality-white.csv', sep=';')
red_wine['type'] = 1  # red wine
white_wine['type'] = 0  # white wine
df = pd.concat([red_wine, white_wine], axis=0)
```

3.1 Feature Descriptions

Each wine sample has 11 physicochemical input features plus the quality score as the target:

Feature	Type	Description

fixed acidity	Numeric	Non-volatile acids (mainly tartaric acid); do not evaporate easily
volatile acidity	Numeric	Acetic acid content high levels produce an unpleasant vinegar taste
citric acid	Numeric	Present in small quantities; adds freshness and flavor complexity
residual sugar	Numeric	Sugar remaining after fermentation; determines sweetness level
chlorides	Numeric	Amount of salt in the wine
free sulfur dioxide	Numeric	SO2 not bound to other molecules; prevents microbial growth and oxidation
total sulfur dioxide	Numeric	Total SO2 present; high amounts are detectable in taste and smell
density	Numeric	Influenced by alcohol and sugar content
pH	Numeric	Acidity/basicity on 0-14 scale; most wines fall between 3-4
sulphates	Numeric	Additive contributing to SO2; antimicrobial and antioxidant properties
alcohol	Numeric	Alcohol percentage by volume
quality	Target	Expert score from 0-10; binarized into Good (>=7) vs Not Good (<7)

3.2 Data Quality Check

After loading, we inspected both the datasets for missing values. The result was reassuring that neither the red wine nor the white wine dataset contained any missing values across all 12 features. This means no imputation was required, and we could proceed directly to feature engineering without data cleaning concerns.

Dataset	Rows	Columns	Missing Values
Red Wine	1,599	13	None (0)
White Wine	4,898	13	None (0)
Combined	6,497	13	None (0)

4. Exploratory Data Analysis (EDA)

Before building any models, we explored the data to understand what we are working with. EDA serves as a reality check; it helps us spot class imbalance, understand feature distributions, and identify which variables might be meaningful predictors.

4.1 Quality Score Distribution

Looking at the raw quality scores (before binarization), we can see:

- Scores range from 3 to 9; notably, no wine received a perfect score of 10 or a truly terrible score of 0 or 1
- Score 6 is by far the most common (~2,800 wines), followed by score 5 (~2,100)
- Scores at the extremes (3 and 9) are very rare, making them statistically challenging
- The distribution is roughly bell-shaped but slightly left-skewed

4.2 Binary Class Distribution & Class Imbalance

The most important finding from EDA was the significant class imbalance in the target variable. When we binarized quality scores into “Good” vs. “Not Good wine”, only 19.7% of all wines qualified as “Good” (score >= 7), while 80.3% were “Not Good”.

Class	Count	Percentage
Not Good (quality < 7)	~5,211	80.3%
Good (quality >= 7)	~1,286	19.7%

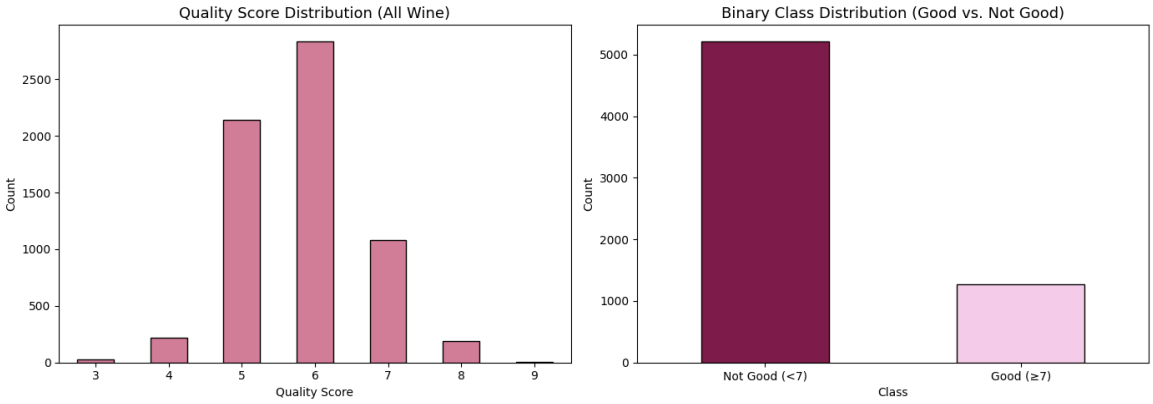

Figure 1: Quality Score Distribution (left) and Binary Class Distribution (right)

This class imbalance can make the model biased toward the majority class. For example, a model could simply label every wine as “Not Good” and still get 80% accuracy, but that wouldn’t make it useful at all. That’s why we did not rely on accuracy alone. Instead, we used AUC-ROC as our main evaluation metric because it shows how well the model can separate the two classes overall, even when the data is unbalanced.

4.3 Feature Distributions

A 3x4 grid of histograms was plotted for all numeric features. Key observations:

- **Fixed acidity:** roughly bell-shaped with a slight right skew; most wines cluster around 6–8.
- **Volatile acidity:** right-skewed; most wines fall between about 0.2–0.5 with a few higher values.
- **Citric acid:** slightly right-skewed; most wines cluster around 0.2–0.4.
- **Residual sugar:** highly right-skewed, with most wines having low sugar and a long tail of sweeter wines.

- **Chlorides:** strongly right-skewed; most wines have low salt content with a few high outliers.
- **Free sulfur dioxide:** right-skewed; most wines fall between about 10–50 with occasional very high values.
- **Total sulfur dioxide:** right-skewed with a wide spread and some high outliers.
- **Density:** tightly concentrated with a very narrow range around ~0.995–1.000.
- **pH:** roughly normally distributed, centered around ~3.2–3.3.
- **Sulphates:** slightly right-skewed; most wines cluster around 0.4–0.7.
- **Alcohol:** roughly right-skewed; most wines cluster around 9-11% ABV
- **Type:** binary and imbalanced, with significantly more wines in one class than the other.

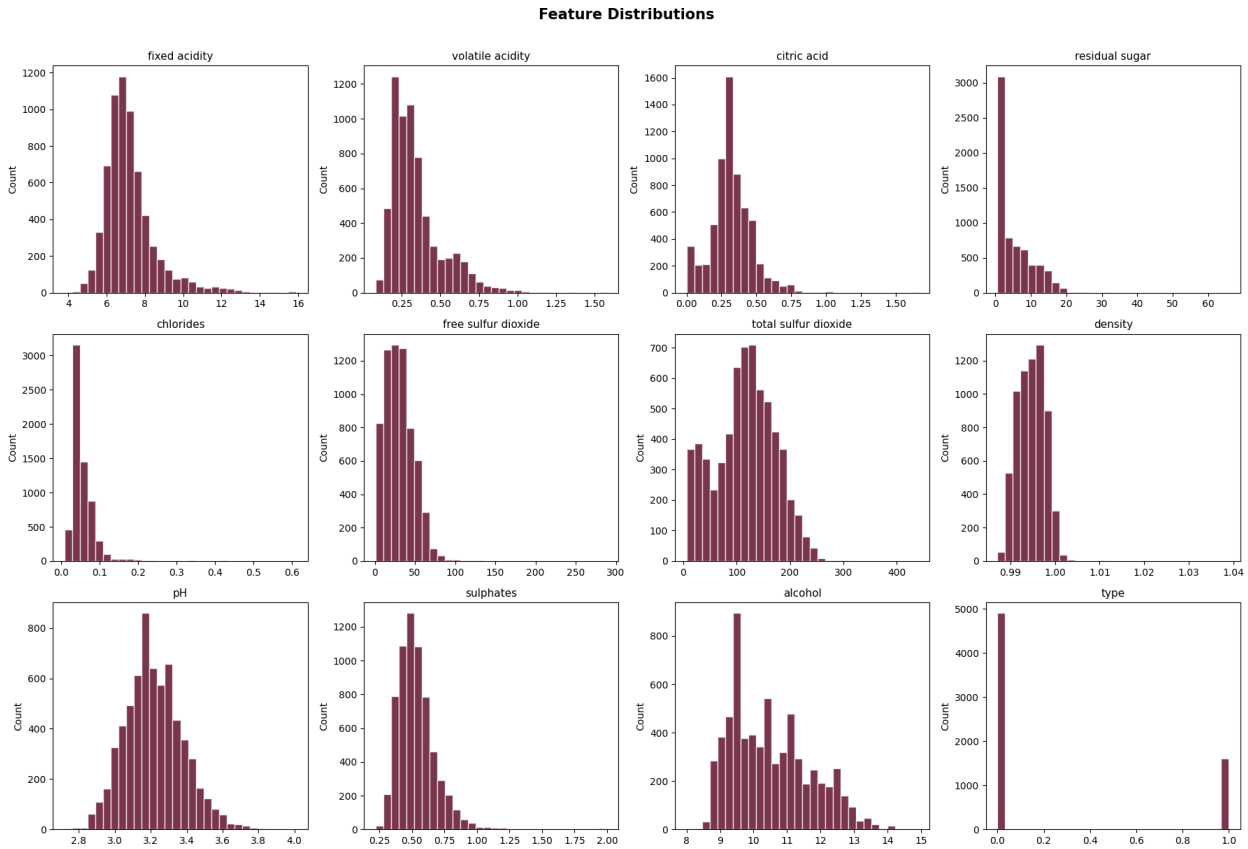

Figure 2: Feature Distributions across all 12 numeric columns

4.4 Correlation Analysis

A lower-triangular correlation heatmap that used the “magma” colormap, revealed the following correlations with wine quality:

Feature	Correlation with Quality	Interpretation
alcohol	+0.444	Alcohol shows the strongest positive correlation with quality in the dataset.
density	-0.306	Negatively correlated; high density tends to mean lower quality
volatile acidity	-0.266	Vinegar character lowers perceived quality

chlorides	-0.201	Saltier wines tend to score lower
citric acid	+0.086	Weak positive effect adds freshness
fixed acidity	+0.077	Very weak relationship
free sulfur dioxide	+0.055	Minimal effect on quality
total sulfur dioxide	-0.041	Minimal negative effect
sulphates	+0.039	Minimal positive effect
residual sugar	+0.037	Almost no linear relationship with quality
pH	-0.020	Essentially uncorrelated with quality

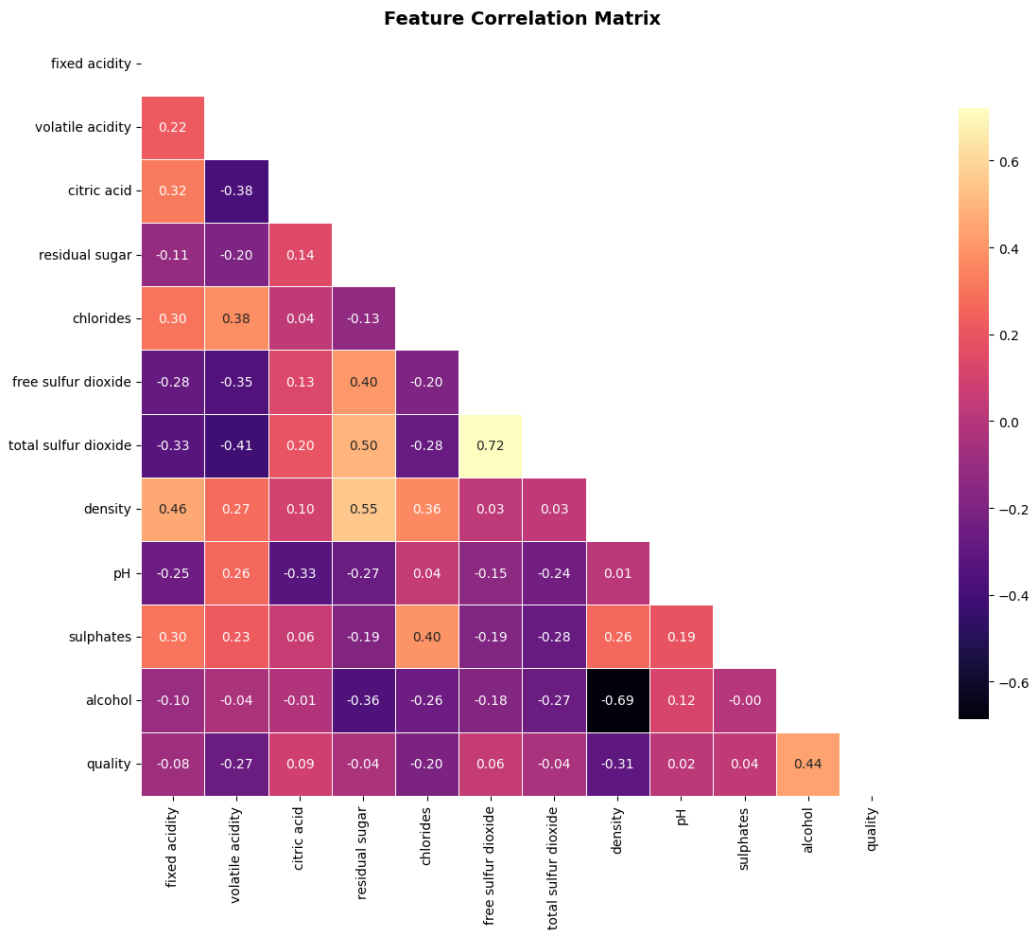

Figure 3: Feature Correlation Matrix (magma colormap)

We observed a high correlation between density and alcohol content. While this can cause multicollinearity issues in simple models, our use of L1/L2 regularization in the modeling phase helped mitigate this by penalizing redundant features.

4.5 EDA Key Takeaways

- **Class imbalance exists:** Only ~20% of wines are classified as “Good” (quality ≥ 7). This is why we use AUC-ROC as our primary metric rather than accuracy.

- **Alcohol is the strongest predictor:** Higher alcohol content correlates positively with quality.
- **Volatile acidity is negatively correlated:** Higher volatile acidity (vinegar taste) tends to mean lower quality.
- **Density and alcohol are strongly negatively correlated** with each other, which makes physical sense.
- **No missing values** were found in either dataset.

5. Data Preparation & Feature Engineering

After exploring the data, we made a few thoughtful preprocessing and feature engineering decisions to get it ready for modeling and below is the explanation for each decision.

5.1 Feature Engineering

Three new features were created to give the models richer information:

New Feature	Formula	Rationale
sulphur_ratio	free SO2 / total SO2	Measures what fraction of total SO2 is “active” (free) versus bound to other molecules. A higher ratio means more protective SO2 is available.
total_acidity	fixed acidity + volatile acidity	Combines both acidity measures into a single signal representing overall acidity, which may be what experts perceive holistically.
body_index	alcohol x residual sugar	Attempts to capture the perceived “body” of a wine, since both higher alcohol and higher residual sugar contribute to a fuller mouthfeel.

After creating sulphur_ratio, any rows where total sulfur dioxide was 0 would have produced division-by-zero (infinity). These infinities, and any resulting NaN values, were replaced with 0 using:

```
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.fillna(0, inplace=True)
```

We created these additional features using a combination of domain knowledge and exploratory intuition. In wine science, factors like sulphur composition, acidity_balance, and body_index are known to influence preservation, taste and mouthfeel. Based on this understanding, we have engineered these features to better capture the underlying relationships in our dataset.

Additionally, we have also retained the original features alongside these engineered ones to ensure that no potentially useful raw information was lost. This allows the model to learn from both the original variables and the newly derived signals. Retaining the original features also helps us understand their independent relationship with wine quality.

5.2 Target Variable Binarization

```
df['is_good'] = (df['quality'] >= 7).astype(int)
```

This turns the original 0–10 quality prediction problem into a simpler binary classification task. We labeled wines with a score of 7 or higher as “good,” since that is a reasonable cutoff on a 10-point

scale and matches how this dataset is often handled in past research. After that, we removed the original quality column from the features.

5.3 Train/Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

The data is split into two groups: 80% as train set and 20% as test set. The critical parameter here is *stratify=y*; this ensures that the 19.7% / 80.3% class ratio is preserved in both the training and test sets. Without stratification, we could accidentally end up with a test set that has very few or very many “Good” wines, making evaluation unreliable.

5.4 Handling Class Imbalance with SMOTE

With only 19.7% of samples being “Good” wines, standard training would result in models biased towards always predicting “Not Good.” We applied SMOTE (Synthetic Minority Oversampling Technique) to address this.

SMOTE works by generating synthetic new examples of the minority class (“Good” wine) by interpolating between existing minority examples in the feature space, not by simply duplicating them. This gives the model a more balanced training set without losing information from the majority class.

```
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)
```

Critically, SMOTE was applied **ONLY** to the training data. The test set was left completely untouched. This is essential because applying oversampling to the test set would create an unrealistically easy evaluation scenario that does not reflect real-world conditions.

5.5 Feature Scaling

```
scaler = StandardScaler()
X_train_scaled_sm = scaler.fit_transform(X_train_sm)
X_test_scaled = scaler.transform(X_test)
```

The StandardScaler was fitted on the SMOTE-resampled training data and then applied to the test set using transform only. Scaling was performed only for Logistic Regression. Decision Tree, Gradient Boosting, and XGBoost were not scaled because tree-based models are inherently scale-invariant and split on feature thresholds, rather than feature magnitudes.

To prevent data leakage, we calculated the scaling parameters (mean and standard deviation) only on the training set and applied them to the test set.

5.6 Summary of Preparation Decisions

Step	Decision	Reason
Combining datasets	Merged red and white with a “type” flag	More training data, and wine type itself may be informative
Feature engineering	Added 3 derived features	Capture chemical relationships not visible in raw features
Target binarization	quality >= 7 = Good	Converts regression target to classification; threshold chosen as natural “good” cutoff

Infinity handling	Replace with 0	Prevents division-by-zero from sulphur_ratio breaking the model
Train/test split	80/20 with stratify=y	Ensures balanced class representation in both sets
SMOTE (training only)	Oversample minority class	Counters 80/20 class imbalance without contaminating test evaluation
StandardScaler	Applied for LR only	LR is sensitive to feature scale; tree models are not

Resulting dataset shape: After preparation, the dataframe has 6,497 rows and 14 features.

6. Model Training

Four classification models were trained and tuned. Each was selected to represent a different modeling approach from simple linear models to complex ensemble methods, allowing us to understand how model complexity affects performance on this problem. All models were tuned using GridSearchCV with 10-fold cross-validation, and scored by AUC-ROC. This means for every hyperparameter combination, the model was trained and validated 10 times on different subsets of the training data, and the average AUC was used to select the best parameters.

6.1 Why These Four Models?

Model	Category	Reason for Including
Logistic Regression	Linear	Simple, interpretable baseline. Good for understanding which features have a direct linear relationship with quality. Requires scaled inputs.
Decision Tree	Non-linear / Tree	Interpretable and can capture non-linear patterns. Prone to overfitting without depth limits. Provides intuitive feature importance.
Gradient Boosting	Ensemble / Boosting	Builds trees sequentially each tree corrects the errors of the previous one. Powerful but slower to train.
XGBoost	Ensemble / Boosting	Optimized, regularized gradient boosting. Typically, state-of-the-art on tabular data. Includes built-in handling for class imbalance via scale_pos_weight.

6.2 Model 1: Decision Tree

Decision Trees can capture non-linear relationships and interactions between chemical features, which makes them well suited for modeling complex patterns in wine quality. This model was tuned with the following hyperparameters:

- **max_depth:** [3, 5, 8, 12] controls how deep the tree grows; deeper trees risk overfitting
- **min_samples_leaf:** [1, 5, 10] minimum number of samples required at a leaf node; higher values enforce generalization
- **criterion:** [gini, entropy] which specifies the function used to evaluate the quality of a split

Best parameters found:

```
criterion: entropy | max_depth: 12 | min_samples_leaf: 10
```

Best CV AUC Score: 0.8899

The grid search selected **entropy** as the splitting criterion along with **a maximum depth of 12** and **min_samples_leaf of 10**. The relatively larger leaf size helps prevent overly specific splits and improves generalization.

6.3 Model 2: Logistic Regression

Logistic Regression was trained on the SMOTE-resampled and scaled training data. The model was tuned using 10-fold cross-validation with ROC-AUC as the scoring metric. The following hyperparameters were tuned:

- **C:** [0.01, 0.1, 1, 10, 100] controls the inverse of regularization strength; smaller C = stronger regularization
- **penalty:** [l1, l2] which specifies the type of regularization; L1 produces sparse models (some coefficients become exactly 0); L2 shrinks all coefficients
- **solver:** liblinear was used as the solver because it's the only solver that supports both L1 and L2 penalties for binary classification

Best parameters found:

```
C: 1 | penalty: l1 | solver: liblinear
```

Best CV AUC Score: 0.8236

The use of L1 regularization with **C = 1** indicates a moderate level of regularization. **L1** regularization performs implicit feature selection by shrinking some coefficients to exactly zero, allowing the model to identify the most informative features. Since Logistic Regression is an interpretable model, the resulting coefficients also provide insight into how different chemical properties influence the probability of a wine being classified as "good."

6.4 Model 3: Gradient Boosting

Gradient Boosting was trained on the SMOTE-resampled training data. Hyperparameter tuning was performed using 10-fold cross-validation with ROC-AUC as the scoring metric. The following hyperparameters were tuned:

- **n_estimators:** [50, 100, 150] which specifies the number of boosting stages (trees) to build
- **learning_rate:** [0.05, 0.1] which controls the contribution of each tree; smaller values apply more shrinkage and typically require more trees
- **max_depth:** [3, 5] which controls the depth of each individual tree in the ensemble

Best parameters found:

```
n_estimators: 150 | learning_rate: 0.1 | max_depth: 5
```

Best CV AUC Score: 0.9676

The selected combination of **150 trees**, a **learning rate of 0.1**, and **max_depth of 5** suggests that strong predictive performance was achieved by combining a relatively large number of moderately complex trees. Since Gradient Boosting builds trees sequentially to correct earlier errors, it is well suited for capturing complex non-linear relationships in the wine quality data.

6.5 Model 4: XGBoost (with Dual Strategy Comparison)

XGBoost was trained using two different strategies for handling class imbalance.

The model was first trained on the original imbalanced data using scale_pos_weight, which internally increases the weight of the minority class. It was then trained on SMOTE-resampled data,

similar to the other models. The two approaches were evaluated on the test set and the better performing strategy was retained.

Strategy	Approach	Test AUC
Method 1: scale_pos_weight	Trains on original imbalanced data; internally upweights minority class by a factor equal to (Not Good count / Good count)	0.9006
Method 2: SMOTE	Trains on SMOTE-resampled balanced data (same as other models)	0.8944

The scale_pos_weight method won, so XGBoost was kept with:

```
learning_rate: 0.1 | max_depth: 6 | n_estimators: 200 | scale_pos_weight: 1
```

Best CV AUC Score: 0.9028 | Test AUC Score: 0.9006

Although the scale_pos_weight strategy was selected, the optimal parameter value was 1, meaning that no additional class weighting was applied. This suggests that XGBoost was able to learn the class patterns effectively without needing extra weighting for the minority class.

7. Model Evaluation & Results

All four models were evaluated on the same held-out test set, which represents 20% of the data (1,300 samples) that the models did not see during training or tuning. Since the dataset was imbalanced (approximately 80% “Not Good” and 20% “Good”), AUC-ROC was used as the primary evaluation metric rather than accuracy. AUC-ROC measures how well a model can distinguish between the two classes across all possible classification thresholds.

The final test-set AUC scores for each model are shown below.

Models	Test AUC
Decision Tree	0.8152
Logistic Regression	0.8042
Gradient Boosting	0.8829
XGBoost	0.9006

Among the four models, **XGBoost** achieved the highest test performance with an AUC of 0.9006, followed by Gradient Boosting (0.8830), Decision Tree (0.8152), and Logistic Regression (0.8043). This ranking indicates that ensemble methods outperformed the simpler single-tree and linear models, suggesting that wine quality depends on complex and non-linear relationships among the chemical features.

7.1 CV Score vs Test Score Comparison

Rank	Model	CV AUC (Train)	Test AUC	Gap
1	XGBoost	0.9028	0.9006	0.0022
2	Gradient Boosting	0.9676	0.8830	0.0846

3	Decision Tree	0.8899	0.8152	0.0747
4	Logistic Regression	0.8236	0.8043	0.0193

To better understand generalization performance, the cross-validated training AUC (from GridSearchCV) was compared with the final test AUC for each model.

XGBoost achieved the highest test AUC (0.9006) and also showed the smallest gap between CV AUC and test AUC (0.0022), indicating the most stable generalization to unseen data. Gradient Boosting also performed strongly, but its larger gap (0.0846) suggests more drop-off from cross-validation to test performance. This difference may partly be explained by the use of SMOTE during training, which introduces synthetic samples that can sometimes make cross-validation performance appear slightly optimistic.

The Decision Tree model showed a moderate gap (0.0747), while Logistic Regression had the lowest overall performance, even though its gap (0.0193) was relatively small.

Overall, the results suggest that ensemble tree-based models, especially XGBoost, were better able to capture the underlying patterns in wine quality while maintaining strong generalization on the test set.

7.2 Best Model: Detailed Classification Report (XGBoost)

Looking at the full classification report for XGBoost on the test set:

Class	Precision	Recall	F1-Score	Support
Not Good (0)	0.91	0.94	0.92	1,044
Good (1)	0.72	0.60	0.65	256
Overall Accuracy	-	-	0.87	1,300
Macro Average	0.81	0.77	0.79	1,300
Weighted Average	0.87	0.87	0.87	1,300

The model performed very well on the majority “Not Good” class, achieving an F1-score of 0.92, while performance on the minority “Good” class was lower, with an F1-score of 0.65. This difference is expected because the dataset remains imbalanced, meaning the model encounters fewer positive examples during training.

In particular, the recall for “Good” wines was 0.60, meaning the model correctly identified about 60% of the actual good wines while missing the remaining 40%. This behavior is common in imbalanced classification problems, where models tend to favor the majority class.

The confusion matrix provides a clearer picture of these results. Most “Not Good” wines were correctly classified (984 out of 1,044), while 153 out of 256 actual good wines were correctly identified. The remaining good wines were predicted as “Not Good,” which reflects the lower recall for the minority class.

For a consumer-facing app, we prioritized **Precision**. It is better for the app to miss a high-quality wine (False Negative) than to recommend a low-quality wine as 'Good' (False Positive), as the latter would damage user trust.

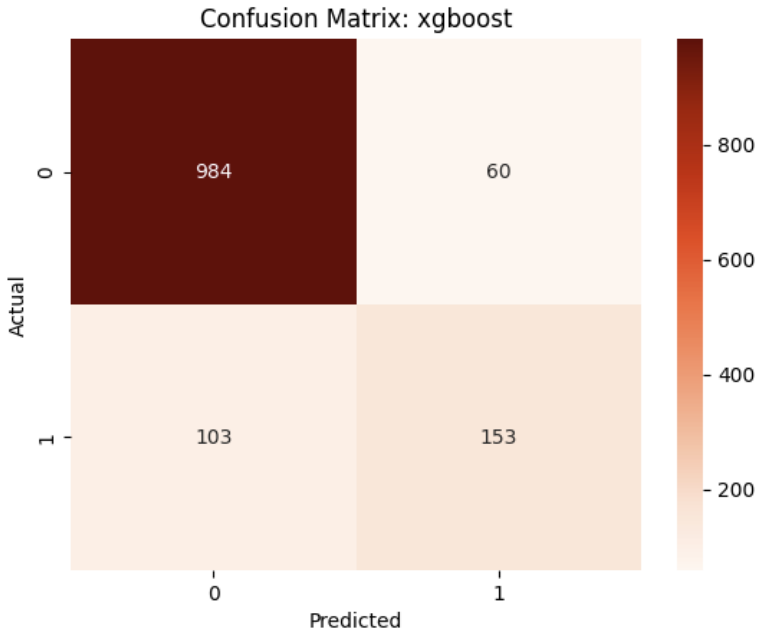

Figure 4: Confusion Matrix for XGBoost (best model)

7.3 ROC Curve Analysis

The ROC curves compare how each model balances true positive rate (sensitivity) and false positive rate across all classification thresholds. A perfect classifier would achieve an AUC of 1.0, while random guessing corresponds to an AUC of 0.5.

- **XGBoost (AUC = 0.90):** shows the strongest performance, maintaining a high true positive rate while keeping false positives relatively low across thresholds.
- **Gradient Boosting (AUC = 0.88):** performs similarly but remains slightly below XGBoost across most parts of the curve.
- **Decision Tree (AUC = 0.82):** performs noticeably worse, with the characteristic “staircase” shape typical of tree-based classifiers.
- **Logistic Regression (AUC = 0.80):** has the smoothest curve due to its probabilistic nature but achieves the lowest AUC overall.

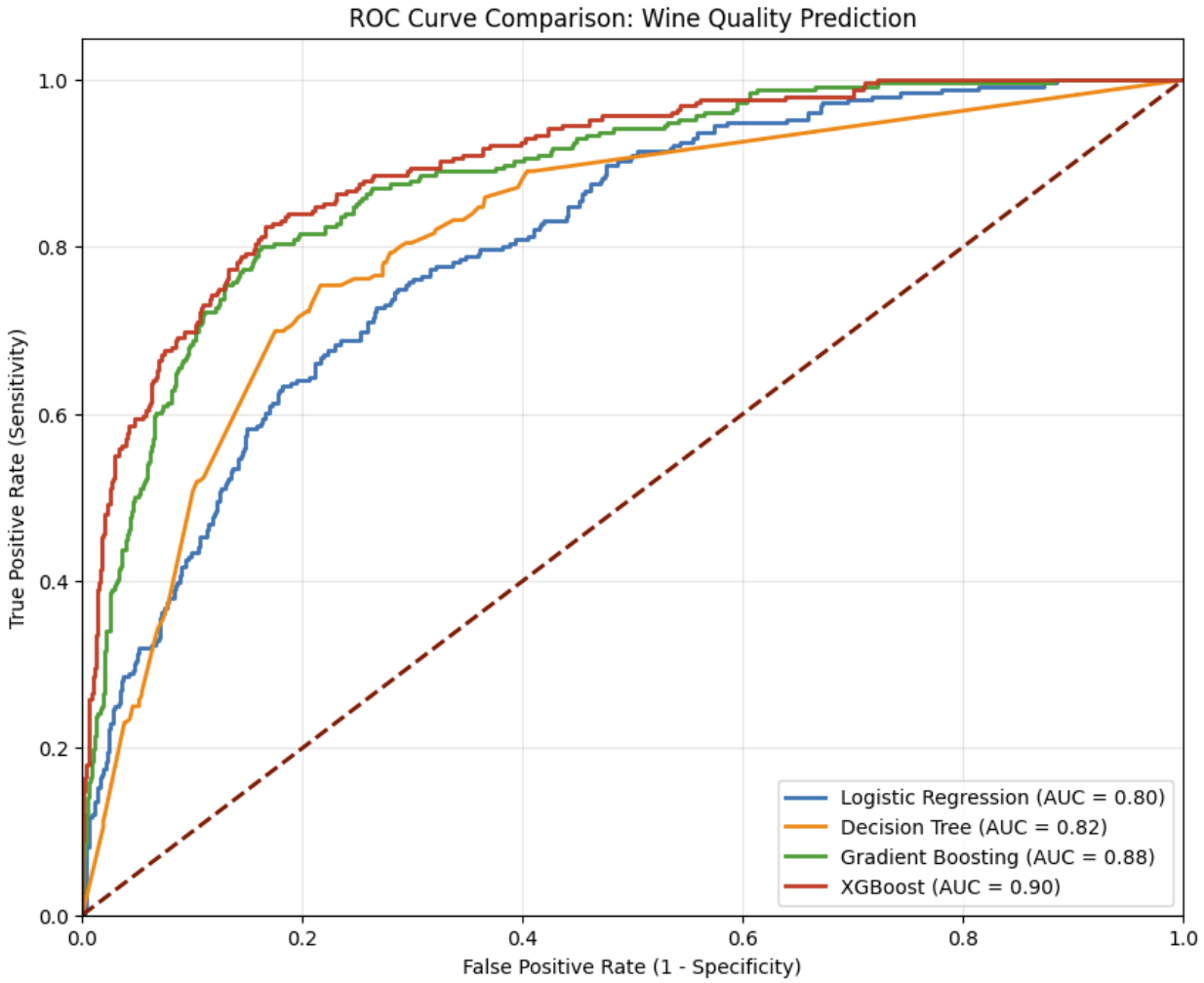

Figure 5: ROC Curve Comparison across all four models

7.4 Feature Importance Analysis

Feature importance analysis was used to examine which variables contributed most to each model's predictions. The graphs below show the feature importance plots for the tree-based models and the coefficient magnitudes for Logistic Regression.

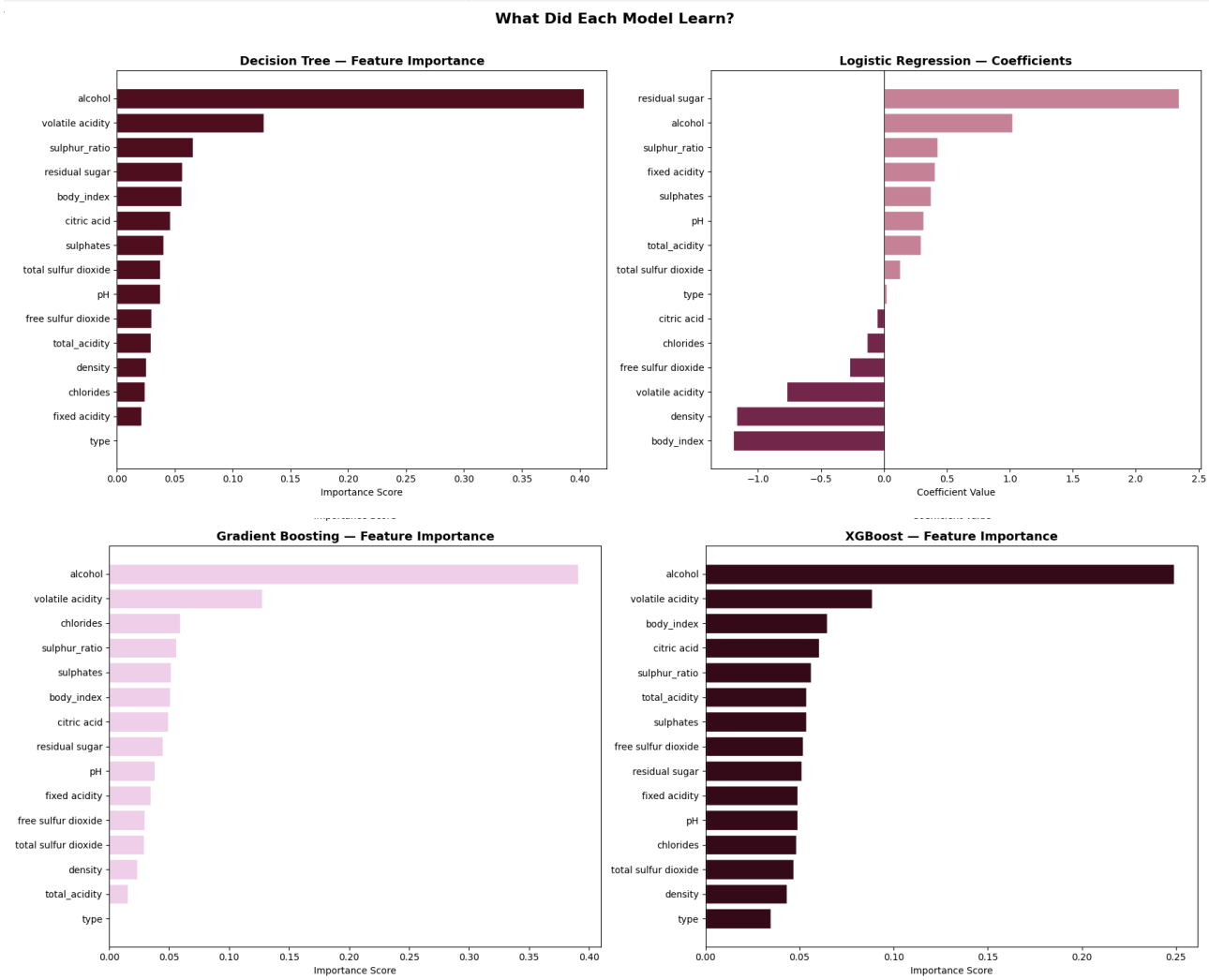

Figure 6: Feature Importance and Coefficients for all four models

The main feature patterns are summarized in the table:

Feature	DT Rank	LR Signal	GB Rank	XGB Rank	Takeaway
alcohol	#1	Strong +	#1	#1	Universally the most important predictor of quality
volatile acidity	#2	Strong -	#2	#2	Negatively impacts quality across all models
density	Mid	Moderate -	#3	#3	Correlated with alcohol; picks up similar signal
sulphates	Low	Moderate +	Mid	Mid	More useful to ensemble methods than simpler models
citric acid	Low	Weak +	Mid	Mid	Adds freshness signal; stronger in boosting models
sulphur_ratio	Low	Weak	Low	Low	Engineered feature; modest contribution

total_acidity	Low	Moderate	Low	Low	Engineered feature; some signal in LR
body_index	Low	Weak	Low	Low	Engineered feature; limited additional value over raw features

Across all four models, **alcohol** consistently appears as the most influential predictor of wine quality. **Volatile acidity** is also highly important and shows a strong negative relationship with wine quality in the Logistic Regression model. This indicates that wines with higher alcohol content are more likely to be classified as “Good,” while higher volatile acidity decreases the probability of a wine being considered good.

The models differ in how broadly they use the available features. **The Decision Tree** concentrates most of its importance on only a few variables, particularly alcohol and volatile acidity, reflecting the greedy splitting behavior of single-tree models. In contrast, **Gradient Boosting** and **XGBoost** distribute importance across a larger set of features, suggesting that ensemble models capture more subtle signals and interactions among the chemical attributes. This difference reflects the ability of ensemble methods to capture interactions between multiple variables.

Engineered variables such as **sulphur_ratio, total_acidity**, and **body_index** appear with smaller importance values. These features provide modest additional signals but do not dominate the predictions compared to the primary chemical attributes.

7.5 One-Line Takeaway Per Model

- **Decision Tree:** Relies heavily on just one or two features (primarily alcohol and volatile acidity), with limited use of the remaining variables due to its greedy splitting behavior.
- **Logistic Regression:** Coefficient signs indicate direction of influence. Positive coefficients increase the probability of predicting a “Good” wine, while negative coefficients decrease it. Alcohol has a strong positive effect, while volatile acidity has a strong negative effect.
- **Gradient Boosting:** Distributes importance more evenly across features than the Decision Tree, indicating that the model captures additional chemical signals such as sulphates and citric acid.
- **XGBoost:** Similar to Gradient Boosting but places stronger emphasis on the top few predictors. Alcohol clearly dominates, reinforcing it as the strongest signal of wine quality across all models.

7.6 What the Results Tell Us

The performance gap between the ensemble models (**XGBoost and Gradient Boosting**) and the simpler models (**Decision Tree and Logistic Regression**) suggests that wine quality is **not** purely a linear problem. Instead, it likely depends on complex interactions among multiple chemical properties.

Additionally, the relatively small differences between cross-validation AUC scores and test AUC scores across models indicate that the models generalize reasonably well to unseen data. This suggests that the relationships learned during model training should extend to new wine samples from the same region without substantial overfitting.

8. Wine Quality Predictor Function

The notebook also includes a **predict_wine_quality()** function that allows anyone to input the chemical properties of a new wine and get a prediction from the best trained model.

```
def predict_wine_quality(fixed_acidity, volatile_acidity, citric_acid,
                         residual_sugar, chlorides, free_sulfur_dioxide,
                         total_sulfur_dioxide, density, pH,
                         sulphates, alcohol, wine_type=0):
```

The function:

- Accepts all **11 physicochemical features** of a wine along with the wine type (0 = white, 1 = red).
- Internally computes the 3 engineered features (**sulphur_ratio, total_acidity, body_index**) to better represent the wine's chemical structure.
- Automatically selects the best-performing model based on test AUC.
- Returns a prediction (Good/Not Good) and a confidence probability.

Example prediction run in the notebook:

```
predict_wine_quality(fixed_acidity=7.0, volatile_acidity=0.27,
citric_acid=0.36,
    residual_sugar=2.0, chlorides=0.045, free_sulfur_dioxide=30,
    total_sulfur_dioxide=120, density=0.994, pH=3.2,
    sulphates=0.6, alcohol=12.5, wine_type=0)
```

Output:

```
Model used: xgboost
Prediction: Not Good Quality
Confidence: 22.9% probability of being Good
```

Even though the wine shows moderate alcohol (12.5%) and relatively low volatile acidity, the overall chemical profile does not align strongly with the patterns associated with high-quality wines in the training data, resulting in a relatively low predicted probability of being classified as "Good."

9. Conclusions & Key Findings

9.1 What We Found

This project shows that wine quality can be predicted reasonably well using only chemical measurements. The best model, XGBoost, achieved a Test AUC of 0.9006, indicating strong ability to distinguish higher-quality wines from lower-quality ones.

Finding	Detail
Alcohol is king	Across all models, alcohol content emerged as the most influential indicator of wine quality. Wines with higher alcohol levels were consistently associated with higher quality scores.
Volatile acidity is the enemy	Wines with higher volatile acidity, which is often associated with a vinegar-like taste, were much less likely to be classified as high quality.

Non-linearity matters	Ensemble models like XGBoost and Gradient Boosting clearly outperformed simpler models such as Logistic Regression and Decision Trees. This suggests that wine quality depends on more complex interactions between features.
Wine type contributes	The type of wine (red vs. white) also contributed useful information, indicating that the two groups have somewhat different chemical patterns and quality distributions.
Good generalization	The difference between cross-validation and test performance was very small, which suggests that the model is capturing meaningful patterns rather than simply memorizing the training data.

9.2 Limitations

We acknowledge several limitations in this analysis:

- **Subjective quality labels:** Wine quality scores were assigned by expert tasting panels. Because sensory evaluation can vary between individuals, the target variable inherently contains some noise.
- **Arbitrary classification threshold:** The “Good” threshold (>=7) is arbitrary: Shifting it to >=6 or >=8 would dramatically change the class distribution and model results. We chose >=7 as a reasonable cutoff, but it is not an objective scientific truth.
- **Class imbalance:** High-quality wines are less common in the dataset, making them harder to detect accurately even after applying imbalance-handling techniques.
- **Geographic restriction:** The data comes exclusively from wines produced in the Vinho Verde region of Portugal, meaning the model may not generalize well to wines from other regions with different production methods and chemical characteristics.

9.3 Future Directions

There are several ways this project could be extended in the future.

- One interesting direction would be to implement SHAP (SHapley Additive exPlanations) to move from global feature importance to **local interpretability**. This would allow the app to explain why a specific bottle received its rating
- Another improvement would be to predict the **full wine quality score** rather than converting it into a binary Good/Not Good label. Treating the problem as an ordinal or multi-class prediction task might capture more information about quality differences.
- It would also be valuable to test the model on **wine datasets from other regions** to see whether the patterns learned here apply more broadly or are specific to Vinho Verde wines.
- Finally, the prediction function could be turned into **a simple web application or API,** allowing winemakers or analysts to enter chemical analysis results and receive instant quality predictions.

9.4 Final Takeaway

Overall, this project demonstrates how machine learning can extract meaningful patterns from chemical measurements to estimate wine quality. While the model cannot replace expert sensory evaluation, it highlights how data-driven approaches can support quality evaluation and early screening in the wine production process. More broadly, this work illustrates how combining domain knowledge with machine learning techniques can provide useful insights even in problems that involve subjective human judgment.